フォーマットファイル一覧テーブル UI仕様書

1. 概要

1.1 コンポーネントの目的

フォーマットファイル一覧テーブルは、選択されたフォーマットフォルダ配下のファイルとサブフォルダをテーブル形式で表示するコンポーネントです。各ファイルに対して編集・ダウンロード・削除の操作ボタンを提供し、フォルダのクリックで階層移動が可能です。

1.2 使用場面

- **表示場所**: フォーマット管理画面の右側パネル（ファイル一覧エリア）
- **主な用途**:
  - フォーマットファイル（XML/MAIML）の一覧表示
  - サブフォルダの一覧表示と階層移動
  - ファイルの編集・ダウンロード・削除操作
  - ファイル名、サイズ、更新日でのソート

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 テーブル構造

```
┌──────┬──────────────────────────────┬──────────┬──────────────┬──────────────────────┐
| タイプ| ファイル名                   | サイズ   | 最終更新日   | 操作                 |
```

```
┌──────┬──────────────────────────┬─────────┬────────────┬──────────────────────┐
│ 📁   │ ↑ 戻る                   │         │            │                      │
├──────┼──────────────────────────┼─────────┼────────────┼──────────────────────┤
│ 📁   │ subfolder (3)            │         │            │                      │
├──────┼──────────────────────────┼─────────┼────────────┼──────────────────────┤
│ 📄   │ material_template.xml    │ 5.2 KB  │ 2時間前    │ [編集][DL][削除]     │
│      │ 材料情報のテンプレート   │         │            │                      │
├──────┼──────────────────────────┼─────────┼────────────┼──────────────────────┤
│ 📄   │ condition_template.xml   │ 3.8 KB  │ 1日前      │ [編集][DL][削除]     │
└──────┴──────────────────────────┴─────────┴────────────┴──────────────────────┘
```

2.2 カラム構成

2.2.1 タイプ（type）

- **幅**: 50px
- **配置**: 中央揃え
- **内容**:
  - フォルダ: 📁 FolderOpenOutlined（オレンジ色）
  - ファイル: 📄 FileTextOutlined（青色）

2.2.2 ファイル名（name）

- **幅**: 可変（残りのスペース）
- **ソート**: 可能（日本語ロケール対応）
- **内容**:
  - ファイル/フォルダ名（フォルダは太字、オレンジ色）
  - フォルダの場合、括弧内にファイル数を表示（例: `subfolder (3)`）
  - ファイルの場合、説明文を下段に表示（secondary color）
- **Tooltip**: ファイルパスを表示

2.2.3 サイズ（size）

- **幅**: 120px
- **ソート**: 可能

- **内容**:
  - ファイルのみ表示（例: `5.2 KB`, `1.5 MB`）
  - フォルダは非表示

2.2.4 最終更新日（lastModified）

- **幅**: 150px
- **ソート**: 可能
- **内容**:
  - ファイルのみ表示（相対時間形式: `2時間前`, `1日前`）
  - フォルダは非表示

2.2.5 操作（actions）

- **幅**: 200px
- **配置**: 中央揃え
- **内容**:
  - ファイルのみ表示（3つのボタン）
  - フォルダは非表示

2.3 操作ボタン

2.3.1 編集ボタン

- **アイコン**: EditOutlined
- **色**: 緑（success）
- **ラベル**: 「編集」
- **動作**: `onEdit(file)` を呼び出し

2.3.2 ダウンロードボタン

- **アイコン**: DownloadOutlined
- **色**: デフォルト（グレー）
- **ラベル**: 「ダウンロード」
- **動作**: `onDownload(file)` を呼び出し

2.3.3 削除ボタン

- **アイコン**: DeleteOutlined
- **色**: 赤（danger）
- **ラベル**: 「削除」
- **動作**: `onDelete(file)` を呼び出し

2.4 インタラクション

2.4.1 行クリック

- **フォルダ行**: クリックで `onFolderClick(folderId)` を呼び出し、フォルダ内に移動
- **ファイル行**: クリックで `onFileClick(file)` を呼び出し（オプション）
- **カーソル**: フォルダまたはonFileClickが設定されている場合はpointer

2.4.2 ボタンクリック

- **イベント伝播の停止**: `e.stopPropagation()` で行クリックイベントを防止
- 各ボタンは独立して動作

---

3. 機能仕様

3.1 ファイル一覧の表示

3.1.1 テーブルレンダリング

```
<Table
  dataSource={files}
  columns={columns}
  rowKey="id"
  size="small"
  pagination={false}
  locale={{
    emptyText: (
      <Empty
        description="フォーマットファイルがありません"
        image={Empty.PRESENTED_IMAGE_SIMPLE}
      />
    ),
  }}
  onRow={(record) => ({
    onClick: () => {
      if (record.isFolder) {
        onFolderClick?.(record.id);
      } else {
        onFileClick?.(record);
      }
    },
    style: {
      cursor: record.isFolder || onFileClick ? 'pointer' : 'default',
    },
  })}
/>
```

3.1.2 UI動作

- ユーザー操作: テーブルデータを表示
- 画面変化:
  - `files` 配列の内容がテーブルに表示される
  - 空の場合は「フォーマットファイルがありません」と表示
- ページネーション: 無効（全件表示）

3.2 ファイルタイプアイコン

3.2.1 アイコン判定

```
const getFileIcon = (file: FormatFileData) => {
  if (file.isFolder) {
    return <FolderOpenOutlined style={{ fontSize: token.fontSizeLG, color: token.colorWarning
}} />;
  }
  return <FileTextOutlined style={{ fontSize: token.fontSizeLG, color: token.colorInfo }} />;
};
```

- **フォルダ**: `FolderOpenOutlined`（オレンジ色、大きめフォント）
- **ファイル**: `FileTextOutlined`（青色、大きめフォント）

3.2.2 UI動作

- ユーザー操作: なし（自動表示）
- 画面変化: タイプカラムにアイコンが表示される

3.3 ファイル名の表示

3.3.1 ファイル名レンダリング

```
render: (text: string, record: FormatFileData) => (
  <div>
    <Tooltip title={record.path} placement="topLeft">
      <Text
        strong={record.isFolder}
        style={{
          color: record.isFolder ? token.colorWarning : 'inherit',
          cursor: record.isFolder ? 'pointer' : 'inherit',
        }}
      >
        {text}
        {record.isFolder && record.fileCount !== undefined && ` (${record.fileCount})`}
      </Text>
    </Tooltip>
    {!record.isFolder && record.description && (
      <div style={{ marginTop: 4 }}>
        <Text type="secondary" style={{ fontSize: token.fontSizeSM }}>
          {record.description}
        </Text>
      </div>
```

```
    )}
  </div>
)
```

3.3.2 表示ルール

- **フォルダ**:
  - 太字、オレンジ色
  - ファイル数を括弧内に表示（例: `subfolder (3)`）
  - カーソルがpointer
- **ファイル**:
  - 通常フォント、デフォルト色
  - 説明文がある場合、下段に小さいフォントで表示
- **Tooltip**: ホバー時にファイルパスを表示

3.3.3 UI動作

- ユーザー操作: ファイル名にホバー
- 画面変化: Tooltipでファイルパスが表示される

3.4 ソート機能

3.4.1 ソート設定

```
// ファイル名ソート
sorter: (a, b) => a.name.localeCompare(b.name, 'ja'),

// サイズソート
sorter: (a, b) => a.size - b.size,

// 更新日ソート
sorter: (a, b) => new Date(a.lastModified).getTime() - new Date(b.lastModified).getTime(),
```

- **ファイル名**: 日本語ロケール対応の文字列比較
- **サイズ**: 数値比較
- **更新日**: 日時の数値比較

3.4.2 UI動作

- ユーザー操作: カラムヘッダーをクリック
- 画面変化:
  - カラムヘッダーにソートアイコン（↑/↓）が表示される
  - テーブルが指定カラムでソートされる
  - 再度クリックで昇順⇔降順が切り替わる

3.5 ファイル編集

3.5.1 編集ボタン処理

```
<Button
  type="primary"
  size="small"
  icon={<EditOutlined />}
  style={{ backgroundColor: token.colorSuccess, borderColor: token.colorSuccess }}
  onClick={(e) => {
    e.stopPropagation();
    onEdit?.(record);
    console.log('Edit file:', record.name);
  }}
>
  編集
</Button>
```

3.5.2 UI動作

- ユーザー操作: 「編集」ボタンをクリック
- 画面変化:
  - イベント伝播が停止される（行クリックイベントが発火しない）

    - 親コンポーネントに `onEdit(file)` が通知される
    - コンソールにログが出力される

- 親コンポーネントでの処理: ファイル編集画面（PetriNetEditorまたはFormatEditorDialog）を開く

3.6 ファイルダウンロード

3.6.1 ダウンロードボタン処理

```
<Button
  type="default"
  size="small"
  icon={<DownloadOutlined />}
  onClick={(e) => {
    e.stopPropagation();
    onDownload?.(record);
  }}
>
  ダウンロード
</Button>
```

3.6.2 UI動作

- ユーザー操作:「ダウンロード」ボタンをクリック
- 画面変化:
    - イベント伝播が停止される
    - 親コンポーネントに `onDownload(file)` が通知される
- 親コンポーネントでの処理: ファイルダウンロードAPIを呼び出し

3.7 ファイル削除

3.7.1 削除ボタン処理

```
<Button
  type="default"
  danger
  size="small"
  icon={<DeleteOutlined />}
  onClick={(e) => {
    e.stopPropagation();
    onDelete?.(record);
    console.log('Delete file:', record.name);
  }}
>
  削除
</Button>
```

3.7.2 UI動作

- ユーザー操作: 「削除」ボタンをクリック
- 画面変化:
  - イベント伝播が停止される
  - 親コンポーネントに `onDelete(file)` が通知される
  - コンソールにログが出力される
- 親コンポーネントでの処理: 削除確認モーダルを表示、確認後にファイル削除APIを呼び出し

3.8 フォルダ移動

3.8.1 フォルダクリック処理

```
onRow={(record) => ({
  onClick: () => {
    if (record.isFolder) {
      onFolderClick?.(record.id);
    } else {
      onFileClick?.(record);
    }
  },
  style: {
    cursor: record.isFolder || onFileClick ? 'pointer' : 'default',
```

```
  },
})}
```

3.8.2 UI動作

- ユーザー操作: フォルダ行をクリック
- 画面変化:
  - 親コンポーネントに `onFolderClick(folderId)` が通知される
  - 親コンポーネントで `currentFolderPath` が更新される
  - ファイル一覧が再取得され、サブフォルダの内容が表示される

3.9 空のテーブル表示

3.9.1 空データ処理

```
locale={{
  emptyText: (
    <Empty
      description="フォーマットファイルがありません"
      image={Empty.PRESENTED_IMAGE_SIMPLE}
    />
  ),
}}
```

3.9.2 UI動作

- ユーザー操作: なし
- 画面変化:
  - `files` 配列が空の場合、Emptyコンポーネントが表示される
  - シンプルな画像と「フォーマットファイルがありません」というメッセージが表示される

---

4. データ仕様

4.1 Props定義

```
interface FormatFileListTableProps {
  files: FormatFileData[];                          // 表示するファイル一覧
  onFileClick?: (file: FormatFileData) => void;     // ファイルクリック時のコールバック
  onFolderClick?: (folderId: string) => void;       // フォルダクリック時のコールバック
  onEdit?: (file: FormatFileData) => void;          // 編集ボタンクリック時のコールバック
  onDownload?: (file: FormatFileData) => void;      // ダウンロードボタンクリック時のコールバッ
ク
  onDelete?: (file: FormatFileData) => void;        // 削除ボタンクリック時のコールバック
}
```

Props説明:

Prop名	型	必須	説明
files	FormatFileData[]	✅	表示するファイル・フォルダ一覧
onFileClick	(file: FormatFileData) => void	-	ファイル行がクリックされた時に呼び出されるコールバック（オプション）
onFolderClick	(folderId: string) => void	-	フォルダ行がクリックされた時に呼び出されるコールバック（フォルダ移動用）
onEdit	(file: FormatFileData) => void	-	編集ボタンがクリックされた時に呼び出されるコールバック
onDownload	(file: FormatFileData) => void	-	ダウンロードボタンがクリックされた時に呼び出されるコールバック
onDelete	(file: FormatFileData) => void	-	削除ボタンがクリックされた時に呼び出されるコールバック

4.2 FormatFileData型定義

```
export interface FormatFileData {
  id: string;           // ファイル/フォルダID
  name: string;         // ファイル/フォルダ名
  path: string;         // S3/Swiftパス
```

```
  size: number;          // ファイルサイズ（バイト）
  lastModified: string; // 最終更新日時（ISO 8601形式）
  type: string;          // MIMEタイプまたはファイルタイプ
  version?: string;      // バージョン（オプション）
  description?: string; // 説明（オプション）
  creator?: string;      // 作成者（オプション）
  isFolder?: boolean;    // フォルダフラグ（true=フォルダ、false/undefined=ファイル）
  fileCount?: number;    // フォルダ内のファイル数（オプション、フォルダの場合のみ）
}
```

FormatFileData説明:

フィールド名	型	必須	説明
id	string	✅	ファイル/フォルダの一意識別子（テーブルのrowKeyとして使用）
name	string	✅	表示用のファイル/フォルダ名
path	string	✅	S3/Swiftの完全パス
size	number	✅	ファイルサイズ（バイト単位、フォルダは0）
lastModified	string	✅	最終更新日時（ISO 8601形式文字列）
type	string	✅	MIMEタイプまたはファイルタイプ（例: `application/xml`）
version	string	-	ファイルのバージョン番号
description	string	-	ファイルの説明文（ファイル名の下に表示）
creator	string	-	ファイル作成者のユーザーID
isFolder	boolean	-	フォルダフラグ（`true`の場合フォルダ、`false`または未定義の場合ファイル）
fileCount	number	-	フォルダ内のファイル数（フォルダの場合のみ）

4.3 State定義

このコンポーネントは**ステートレス**で、ローカルStateを持ちません。 全ての状態管理は親コンポーネント（FormatManagement）で行われます。

---

5. API連携

このコンポーネントは直接APIを呼び出しません。 親コンポーネント（FormatManagement）が取得したデータを`files` Propsとして受け取ります。

6. エラーハンドリング

6.1 エラーケース

このコンポーネントは表示専用のため、独自のエラーハンドリングはありません。

以下のケースは親コンポーネントで処理されます：

- ファイル一覧取得失敗
- ファイルダウンロード失敗
- ファイル削除失敗
- データの不整合（nullやundefined）

6.2 表示の制約

6.2.1 必須フィールドの欠損

```
dataSource={files}
rowKey="id"
```

- `files` 配列の各要素に `id` フィールドが必須
- `id` が欠けている場合、Reactのwarningが発生する可能性があります

6.2.2 フォーマッター関数のエラー

```
{formatFileSize(record.size)}
{formatRelativeTime(record.lastModified)}
```

- `formatFileSize()` が不正な値を受け取った場合、エラーまたは不正な表示になる可能性があります

- `formatRelativeTime()` が不正な日付文字列を受け取った場合、`Invalid Date` と表示される可能性があります
- **推奨**: 親コンポーネントでデータバリデーションを実施

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/project/FormatFileListTable.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **ユーティリティ**:
  - `frontend/src/utils/formatters.ts` (formatFileSize, formatRelativeTime)
- **型定義**: このファイル内で定義・export（FormatFileData型）

7.3 依存ライブラリ

- **Ant Design**:
  - Table
  - Button
  - Tooltip
  - Space
  - Empty
  - Typography (Text)
  - theme
- **Ant Design Icons**:

- FileTextOutlined
- FolderOpenOutlined
- DownloadOutlined
- EditOutlined
- DeleteOutlined

7.4 親子関係

```
FormatManagement (親)
  ↓ Props: files, onFileClick, onFolderClick, onEdit, onDownload, onDelete
FormatFileListTable (このコンポーネント)
```

親コンポーネントから受け取るデータ:

- ファイル・フォルダ一覧（現在のフォルダのファイルのみ）

親コンポーネントへの通知:

- ファイルクリックイベント
- フォルダクリックイベント（階層移動）
- 編集イベント
- ダウンロードイベント
- 削除イベント

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 親コンテナの100%
- **高さ**: コンテンツに応じて可変
- **テーブルサイズ**: `size="small"` で省スペース表示

8.2 スクロール

- **縦スクロール**: ページネーション無効のため、テーブル全体が表示される
- 親コンテナでスクロールが管理される

8.3 視覚的フィードバック

8.3.1 ホバー効果

- **行ホバー**: Ant Designのデフォルトホバー効果（背景色が薄い灰色に変わる）
- **カーソル**: フォルダまたはonFileClickが設定されている場合はpointer

8.3.2 フォルダとファイルの区別

- **フォルダ**:
  - アイコン: 📁（オレンジ色）
  - 名前: 太字、オレンジ色
  - ファイル数表示: 括弧内に数字
  - サイズ・更新日・操作ボタン: 非表示

- **ファイル**:
  - アイコン: 📄（青色）
  - 名前: 通常フォント、デフォルト色
  - サイズ・更新日: 表示
  - 操作ボタン: 3つ表示（編集/ダウンロード/削除）

8.3.3 ボタンの色分け

- **編集ボタン**: 緑（success）- ポジティブなアクション
- **ダウンロードボタン**: グレー（default）- 中立的なアクション
- **削除ボタン**: 赤（danger）- 危険なアクション

8.4 アクセシビリティ

- **クリック領域**: 行全体がクリック可能（フォルダ移動またはファイル詳細）
- **ボタン**: 個別にクリック可能、イベント伝播を停止
- **Tooltip**: ファイルパスの詳細情報を提供
- **Empty表示**: データがない場合の明確なメッセージ

8.5 ソート機能

- **デフォルト**: ソートなし（API返却順）
- **ユーザー操作**: カラムヘッダーをクリックでソート
- **視覚的フィードバック**: ソートアイコン（↑/↓）が表示される

---

9. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline